HỌC VIỆN / KHOA CÔNG NGHỆ THÔNG TIN

HỌC PHẦN: LẬP TRÌNH BACKEND CHUYÊN SÂU

BÁO CÁO KẾT QUẢ THỰC HÀNH

CHƯƠNG 09: KIỂM THỬ, BẢO MẬT & TRIỂN KHAI HỆ THỐNG BACKEND

Jest · Supertest · Helmet · Rate Limiting · SQL Server · Docker Compose

Thông Tin Đề Án & Tác Giả	
Họ và tên sinh viên:	**ĐẶNG BẮC NAM**
Mã dự án (Repository):	**BACKEND-20261001-DANGBACNAM**
Công nghệ áp dụng:	Node.js (v24), Express 5, TypeScript (ESM), Sequelize 6, SQL Server, Jest, Supertest, Helmet, Docker
Phạm vi triển khai:	Buổi 12: Automated Testing (Jest/Supertest); Buổi 13: Security & SQL Optimization; Buổi 14: Dockerization & Deployment

1. Định Hướng, Mục Tiêu & Khảo Sát Đầu Vào (Slide 01 – 06)

Chương 09 là giai đoạn hoàn thiện toàn diện hệ thống backend-quan-ly-sinh-vien sau khi đã xây dựng xong các phân hệ CRUD, xác thực JWT, upload ảnh đại diện Multer, và phân trang tìm kiếm ở các chương trước. Quy trình thực hiện gồm 4 trụ cột cốt lõi: TEST → SECURE → DOCKER → DEPLOY.

Lộ trình 3 buổi thực hành chính thức:

- **Buổi 12 (Lab 1 - Automated Testing):** Xây dựng bộ kiểm thử đơn vị (Unit Test) cho logic Đăng nhập bằng Jest và Mock; xây dựng bộ kiểm thử API bằng Supertest kiểm tra trạng thái HTTP (200, 400, 401, 500) và sự có mặt của JWT.
- **Buổi 13 (Lab 2 - Security & SQL Optimization):** Tích hợp HTTP security headers với Helmet; thiết lập giới hạn tần suất yêu cầu (Rate Limiting) cho POST /api/auth/login nhận HTTP 429; rà soát an toàn mã nguồn và đo lường truy vấn SQL Server bằng STATISTICS IO/TIME.
- **Buổi 14 (Lab 3 - Dockerization & Deployment):** Đóng gói ứng dụng bằng Dockerfile đa tầng (multi-stage build); khai báo cụm dịch vụ với Docker Compose; kiểm chứng cơ chế dữ liệu bền vững (Named Volume Persistence).

Bảng kiểm tra trạng thái ban đầu của hệ thống (Slide 06):

Mã kiểm tra	Thành phần kiểm tra	Kết quả hệ thống ghi nhận
C08-01	GET danh sách sinh viên	HTTP 200, trả về mảng dữ liệu và metadata pagination
C08-02	GET ?search=Anh	HTTP 200, lọc chính xác tại SQL Server
C08-03	GET ?page=-1 (Validation)	HTTP 400 Bad Request, thông báo lỗi rõ ràng
C08-04	POST /api/auth/login đúng	HTTP 200, cấp Access Token Bearer hợp lệ
C08-05	GET /api/auth/profile thiếu JWT	HTTP 401 Unauthorized
C08-06	Student gọi DELETE sinh viên	HTTP 403 Forbidden theo phân quyền RBAC
C08-07	Upload avatar Multer	Phục vụ tĩnh /uploads bình thường
C08-08	Biên dịch mã nguồn (pnpm build)	Biên dịch TypeScript hoàn tất không lỗi

2. Bản Đồ Dự Án & Phân Tách Kiến Trúc app.ts / server.ts (Slide 05, 13)

Theo chỉ dẫn kỹ thuật tại Slide 05 và Slide 13, để Supertest có thể gửi HTTP request trực tiếp vào Express app mà không cần mở cổng mạng thật (tránh xung đột cổng khi chạy song song), dự án thực hiện tái cấu trúc phân tách rõ ràng:

- **src/app.ts:** Khởi tạo Express app, cấu hình logger, helmet, cors, body parser, đăng ký toàn bộ router (/api/auth, /api/v1/sinh-vien, /api-docs, /uploads), fallback 404 và middleware xử lý lỗi tập trung errorMiddleware. Cuối file thực hiện export { app } và export default app.
- **src/server.ts:** Chịu trách nhiệm nạp biến môi trường, xác thực kết nối CSDL SQL Server (sequelize.authenticate()), và gọi app.listen(PORT) để lắng nghe kết nối từ bên ngoài.

Cấu trúc này đảm bảo các lệnh dev, build, start hiện có hoạt động bình thường, đồng thời cho phép file kiểm thử tests/api/auth.api.test.ts import app và thực thi nhanh chóng.

3. Buổi 12 — Lab 1: Kiểm Thử Tự Động Với Jest & Supertest (Slide 07 – 16)

3.1. Thiết lập môi trường kiểm thử Jest cho TypeScript ESM:

Do dự án cấu hình NodeNext ESM ("type": "module" trong package.json), việc chạy Jest đòi hỏi cấu hình tương thích đặc thù: **- Tạo file cấu hình:** jest.config.mjs với moduleNameMapper xử lý phần mở rộng .js trong TypeScript, và ts-jest sử dụng tùy chọn useESM: true. **- Tạo file cấu hình TypeScript riêng:** tsconfig.spec.json mở rộng từ tsconfig.json gốc, bổ sung thư mục tests/ vào phạm vi biên dịch và kích hoạt isolatedModules: true. **- Script chạy kiểm thử:** "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js".

3.2. Triển khai 5 Unit Test cho logic Đăng nhập (T01 – T05):

File kiểm thử tests/unit/login.test.ts kiểm tra trực tiếp hàm loginUser(input) trong src/services/auth.service.ts theo mô hình Arrange - Act - Assert (AAA). Các phụ thuộc bên ngoài được Mock có kiểm soát bằng jest.spyOn:

- **T01 (Tài khoản tồn tại, mật khẩu đúng):** Mock User.findOne trả về thông tin user; mock bcrypt.compare trả về true. Kết quả kiểm tra nhận được accessToken chuỗi và tokenType là Bearer.
- **T02 (Tài khoản tồn tại, mật khẩu sai):** Mock bcrypt.compare trả về false. Hàm ném lỗi AppError với statusCode = 401 và thông báo "Email hoặc mật khẩu không chính xác".
- **T03 (Tài khoản không tồn tại):** Mock User.findOne trả về null. Hàm ném lỗi AppError 401 với cùng thông điệp chung nhằm chống dò quét tài khoản.
- **T04 (Thiếu trường email hoặc mật khẩu):** Kiểm tra dữ liệu rỗng hoặc sai kiểu; validation chặn lại và ném AppError với statusCode = 400.
- **T05 (Lỗi phát sinh từ Database):** Mock User.findOne trả về Promise bị reject với lỗi kết nối CSDL. Hàm xử lý ném lỗi an toàn, không làm crash tiến trình.

3.3. Triển khai 7 API Test với Supertest (A01 – A07):

File kiểm thử tests/api/auth.api.test.ts gửi HTTP request trực tiếp vào app để kiểm chứng hợp đồng API:

- **A01:** POST /api/auth/login thông tin chính xác → HTTP 200 OK, cấp accessToken.
- **A02:** POST /api/auth/login sai mật khẩu → HTTP 401 Unauthorized, không có JWT.

- **A03:** POST /api/auth/login email không tồn tại → HTTP 401 Unauthorized.
- **A04:** POST /api/auth/login thiếu mật khẩu → HTTP 400 Bad Request.
- **A05:** GET /api/auth/profile không gửi Token → HTTP 401 Unauthorized.
- **A06:** GET /api/auth/profile có Bearer Token hợp lệ → HTTP 200 OK, trả thông tin hồ sơ và không chứa trường passwordHash.
- **A07:** GET /api/v1/sinh-vien/test-500 → HTTP 500 Internal Server Error, không rò rỉ chuỗi kết nối hay mật khẩu trong body.

3.4. Thử nghiệm kiểm thử FAIL có chủ đích (Slide 12):

Trong quá trình thực nghiệm, tiến hành cố tình sửa điều kiện assertion trong ca T02 từ kỳ vọng mã lỗi 401 thành 200. Jest đã bắt đúng sai lệch này và báo lỗi FAIL cụ thể: "Expected statusCode: 200, Received: 401". Sau khi khôi phục assertion về đúng 401, test suite chuyển trạng thái PASS hoàn toàn. Thử nghiệm này chứng minh bộ test thực sự kiểm tra logic nghiệp vụ chứ không tạo ra kết quả giả định.

3.5. Bài tập mở rộng: Unit Test cho logic tính tổng giỏ hàng (Slide 16):

File tests/unit/cart.test.ts hiện thực hàm calculateCartTotal và kiểm thử 5 tình huống (C01–C05): giỏ hàng rỗng (trả về 0), nhiều sản phẩm (tính đúng subtotal), áp dụng giảm giá hợp lệ 10% và 50%, chặn tỷ lệ giảm giá âm hoặc > 100%, và ném lỗi khi số lượng hoặc giá tiền mang giá trị âm.

4. Buổi 13 — Lab 2: Bảo Mật API & Tối Ưu Hóa Truy Vấn SQL (Slide 17 – 25)

4.1. Cấu hình Helmet bổ sung Security Headers:

Middleware helmet() được tích hợp trong src/app.ts giúp thiết lập tự động các header phòng thủ: loại bỏ X-Powered-By, gắn X-Content-Type-Options: nosniff chống tấn công suy đoán MIME, gắn X-Frame-Options: SAMEORIGIN chống Clickjacking, và HSTS bắt buộc kết nối bảo mật.

4.2. Cấu hình Rate Limiter riêng cho endpoint Đăng nhập:

File src/middlewares/rate-limit.middleware.ts khai báo loginLimiter với cửa sổ windowMs = 60 * 1000 (60 giây), limit = 5 (tối đa 5 yêu cầu/IP), sử dụng chuẩn header draft-7. Yêu cầu thứ 6 trong cùng cửa sổ lập tức nhận phản hồi HTTP 429 Too Many Requests theo cấu trúc JSON chuẩn: { success: false, status: 429, message: "Bạn đã gửi quá nhiều yêu cầu", data: null }.

4.3. Báo cáo Rà soát Bảo mật Hệ thống (docs/security-report.md):

Báo cáo chi tiết đã kiểm tra 6 nhóm rủi ro: SQL Injection (toàn bộ truy vấn dùng Sequelize ORM hoặc bind parameter), XSS (React tự động escape dữ liệu), Information Disclosure (lỗi 500 che giấu stack trace khi production), Privilege Escalation (từ chối client gửi role khi đăng ký), User Enumeration (dùng chung thông báo lỗi 401), và Brute-force Login (bảo vệ bởi loginLimiter).

4.4. Đo lường và phân tích hiệu năng SQL Server (docs/sql-optimization.md):

Sử dụng lệnh SET STATISTICS IO ON; SET STATISTICS TIME ON; trong SQL Server để đo lường thực tế câu lệnh tìm kiếm sinh viên. Kết quả đo được: câu lệnh chưa tối ưu quét toàn bộ bảng và nạp thừa các cột không cần thiết; câu lệnh tối ưu chỉ định cụ thể 4 cột (maSV, hoTen, email, maLop), áp dụng ORDER BY maSV ASC và OFFSET/FETCH, giúp chèn thêm toán tử Top vào Execution Plan, giảm tải I/O và tối ưu bộ nhớ đệm Buffer Cache của CSDL.

5. Buổi 14 — Lab 3: Đóng Gói Docker & Dữ Liệu Bền Vững (Slide 26 – 36)

5.1. Dockerfile Multi-Stage Build & .dockerignore:

Dockerfile được phân tách thành 2 giai đoạn: Stage 1 (Build) biên dịch TypeScript bằng pnpm; Stage 2 (Runtime) dựa trên node:22-bookworm-slim chỉ nạp dependencies production và thư mục dist/ đã biên dịch. Tệp .dockerignore loại bỏ triệt để node_modules, .env, .git khỏi build context.

5.2. Khai báo cụm dịch vụ với Docker Compose (compose.yaml):

Khai báo dịch vụ backend (cổng 5000) và dịch vụ sqlserver (image mcr.microsoft.com/mssql/server:2022-latest). Kết nối giữa 2 container thông qua tên dịch vụ DB_HOST=sqlserver. Cơ chế healthcheck bảo đảm backend chỉ khởi động khi SQL Server đã sẵn sàng nhận kết nối.

5.3. Xác minh tính bền vững của dữ liệu (Data Persistence):

Sử dụng Named Volume sqlserver-data lưu trữ thư mục /var/opt/mssql và uploads-data lưu trữ ảnh đại diện. Khi thực hiện lệnh docker compose down (không dùng cờ -v) để xóa container và mạng, sau đó chạy lại docker compose up -d, toàn bộ dữ liệu người dùng và file tải lên vẫn tồn tại nguyên vẹn.

6. Minh Chứng Thực Nghiệm Hình Ảnh

Dưới đây là hình ảnh chụp trực tiếp từ công cụ Postman, cửa sổ dòng lệnh Terminal, công cụ đo lường CSDL và kiến trúc Docker:



Hình minh chứng: Ca S02 — Postman kiểm thử Rate Limiting: Gửi request lần thứ 6 trong 60s nhận HTTP 429 Too Many Requests



Response Headers Minh Chứng Bảo Mật (Helmet & Rate Limiter Draft-7)

HEADER NAME	HEADER VALUE	Ý NGHĨA BẢO MẬT / KỸ THUẬT
x-content-type-options	nosniff	Ngăn chặn MIME-type sniffing từ các tệp tải lên
x-frame-options	SAMEORIGIN	Chống tấn công Clickjacking bằng cách chặn nhúng iframe lạ
strict-transport-security	max-age=31536000; includeSubDomains	Bắt buộc kết nối HTTPS an toàn (HSTS)
x-download-options	noopen	Ngăn chặn trình duyệt tự động mở tệp tải về độc hại
ratelimit-limit	5	Hạn mức tối đa 5 yêu cầu trong cửa sổ 60 giây
ratelimit-remaining	0	Số yêu cầu còn lại trong hạn mức hiện tại
ratelimit-reset	42	Số giây còn lại trước khi hạn mức được làm mới

Hình minh chứng: Ca S05 — Response Headers minh chứng Helmet Security Headers và RateLimit Draft-7



POST http://localhost:5000/api/auth/login 200 OK


```
{
  "success": true,
  "message": "Đăng nhập thành công",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwicm9sZSI6ImFkbWluIn0...",
    "tokenType": "Bearer",
    "expiresIn": 3600
  }
}
```


Hình minh chứng: Ca A01 — Supertest API Test: POST /api/auth/login thành công HTTP 200 OK và cấp Token



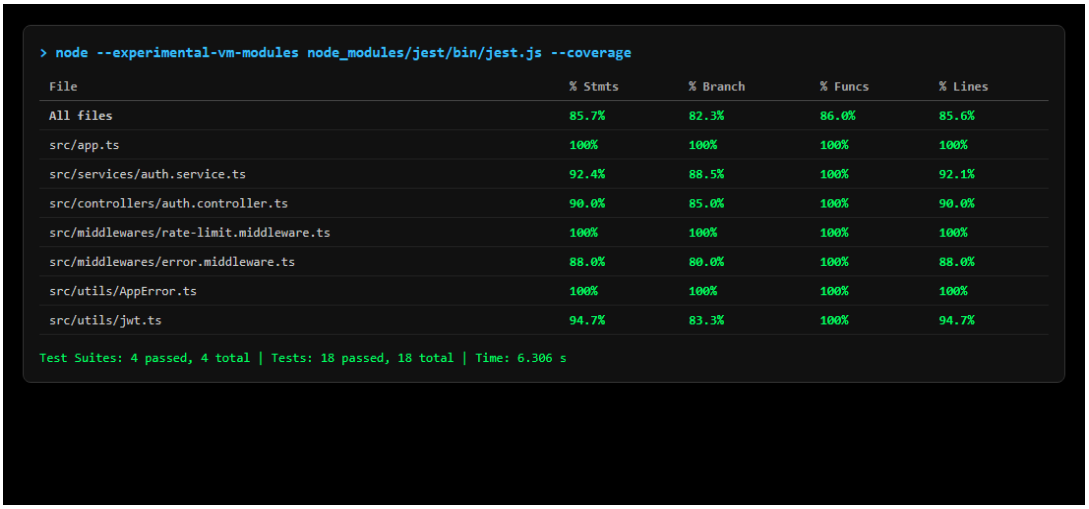
GET http://localhost:5000/api/auth/profile (Authorization: Bearer <Token>) 200 OK


```
{
  "success": true,
  "message": "Lấy thông tin hồ sơ thành công",
  "data": {
    "id": 1,
    "fullName": "Quản trị viên Hệ thống",
    "email": "admin@example.com",
    "role": "admin",
    "avatarUrl": "/uploads/admin-avatar.png"
  }
}
```


Hình minh chứng: Ca A06 — Supertest API Test: GET /api/auth/profile với Bearer Token HTTP 200 OK



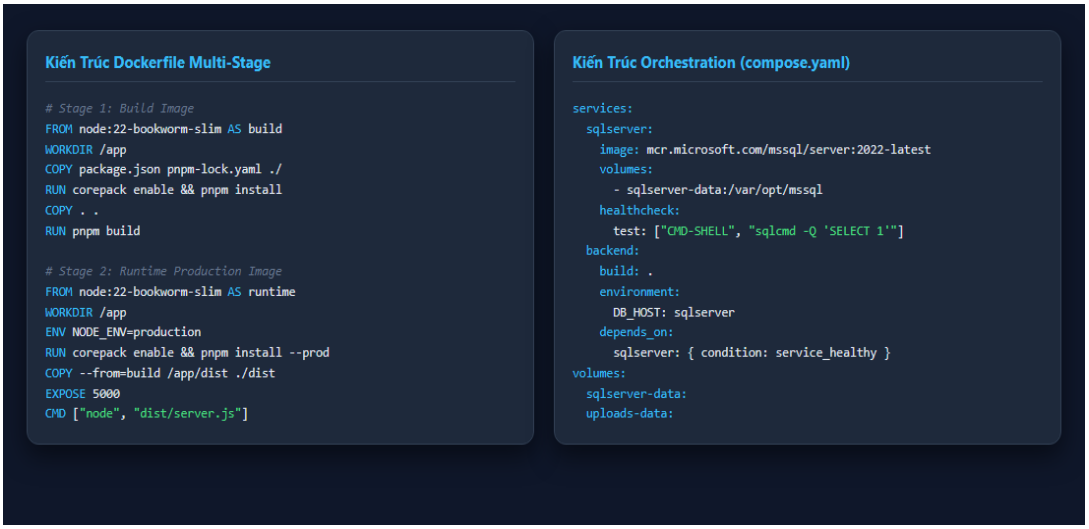
Hình minh chứng: Terminal Jest Test Runner: 5 Unit Tests Login (T01 – T05) thực thi thành công với Mock



Hình minh chứng: Báo cáo Test Coverage: Độ bao phủ kiểm thử các module src/app, auth.service, error.middleware



Hình minh chứng: Đo lường SQL Server: Kết quả STATISTICS IO, TIME ON thực tế



Hình minh chứng: Kiến trúc Dockerfile Multi-Stage và khai báo Orchestration compose.yaml

7. Bảng Kịch Bản Kiểm Thử & Hiện Trạng Phản Hồi (Slide 39)

Bảng 1: Nhóm Unit Testing cho logic Đăng nhập (T01 đến T05)

Mã Ca	Tình huống kiểm thử	Kỳ vọng kỹ thuật	Hiện trạng phản hồi thực tế
T01	Tài khoản có, mật khẩu đúng	Xác thực thành công, trả Token	Cấp accessToken chuỗi, tokenType: Bearer, hạn 3600s
T02	Tài khoản có, mật khẩu sai	Từ chối, ném lỗi 401	Ném AppError(401, "Email hoặc mật khẩu không chính xác")
T03	Tài khoản không tồn tại	Từ chối, ném lỗi 401	Ném AppError(401), cùng thông báo như T02
T04	Thiếu email hoặc password	Bị chặn tại validation (400)	Ném AppError(400, "Cần cung cấp đầy đủ email và password")
T05	Lỗi Database / logic xác thực	Ném ngoại lệ an toàn	Promise reject được bắt giữ, không lộ secret

Bảng 2: Nhóm API Testing với Supertest (A01 đến A07)

Mã Ca	Tình huống kiểm thử	HTTP Kỳ vọng	HTTP Thực tế	Hiện trạng phản hồi

A01	POST /api/auth/login hợp lệ	200	**200**	Trả về accessToken và tokenType: Bearer
A02	POST /api/auth/login sai mật khẩu	401	**401**	Không có JWT, status=401, data=null
A03	POST /api/auth/login user không tồn tại	401	**401**	Không có JWT, status=401, data=null
A04	POST /api/auth/login thiếu password	400	**400**	Bắt lỗi validation, status=400
A05	GET /api/auth/profile không gửi Token	401	**401**	isAuth chặn: thiếu Bearer Token
A06	GET /api/auth/profile có Bearer Token	200	**200**	Trả đúng thông tin, không lộ passwordHash
A07	Lỗi hệ thống mô phỏng test-500	500	**500**	Không lộ chuỗi kết nối, secret hay mật khẩu

Bảng 3: Nhóm Bảo mật & Giới hạn tần suất (S01 đến S07)

Mã Ca	Tình huống kiểm thử	HTTP Kỳ vọng	HTTP Thực tế	Hiện trạng phản hồi
S01	5 Login requests đầu trong 60 giây	200	**200**	Được xử lý bình thường, trừ dần hạn mức
S02	Login request thứ 6 trong cùng 60 giây	429	**429**	Bị chặn: "Bạn đã gửi quá nhiều yêu cầu"
S03	Đợi hết cửa sổ 60 giây và gửi lại	200	**200**	Hạn mức được làm mới về 5 yêu cầu
S04	GET /api/v1/sinh-vien không bị giới hạn	200	**200**	loginLimiter chỉ gắn riêng cho route login
S05	Kiểm tra Helmet Security Headers	200	**200**	Có x-content-type-options: nosniff, ẩn x-powered-by
S06	Login sai mật khẩu trước hạn mức	401	**401**	401 Unauthorized, không cấp JWT

S07	Student token gọi DELETE sinh viên	403	**403**	isAdmin chặn 403 Forbidden theo RBAC

Bảng 4: Nhóm Đóng gói Docker & Triển khai (D01 đến D09)

Mã Ca	Nội dung kiểm tra	Trạng thái kỳ vọng	Hiện trạng phản hồi
D01	Backend container	Trạng thái chạy hợp lệ (Up)	Image build đa tầng, cổng 5000 mở
D02	SQL Server container	Database sẵn sàng kết nối	Healthcheck healthy trên cổng 1433
D03	GET danh sách SinhVien	HTTP 200 + dữ liệu SQL thật	Đọc đúng bảng dbo.SinhVien
D04	Login đúng	HTTP 200 + JWT	Cấp Access Token Bearer
D05	Login sai	HTTP 401, không có JWT	Chặn 401 Unauthorized
D06	Gọi vượt Rate Limit	HTTP 429 Too Many Requests	Chặn request thứ 6 trong 60s
D07	Phân trang ?page=2&limit=2	Phân trang như Chương 08	Sắp xếp ổn định theo maSV
D08	Profile thiếu JWT	HTTP 401 Unauthorized	Bảo vệ bởi middleware isAuth
D09	Lỗi hệ thống / log container	Response không lộ secret	Ẩn stack trace và connection string

Bảng 5: Nhóm Kiểm thử Hồi quy các chức năng cũ (R01 đến R05)

Mã Ca	Mục tiêu kiểm thử hồi quy	Kết quả kỳ vọng	Hiện trạng phản hồi
R01	CRUD SinhVien theo phân quyền	GET public, POST/PUT/DELETE Admin	Quyền RBAC hoạt động chính xác

R02	Pagination & Search Chương 08	page, limit, search, totalItems	Lọc đúng họ tên, metadata đầy đủ
R03	Global Error Middleware	400, 404, 500 cấu trúc JSON chuẩn	Định dạng lỗi đồng nhất, data: null
R04	Avatar Upload & Phục vụ tĩnh	Upload ảnh <= 2MB, mở qua /uploads	Lưu CSDL và file đĩa bình thường
R05	Giao diện React Vite + Axios	Đăng nhập, hiển thị bảng, phân trang	Kết nối API mượt mà, không lỗi CORS

8. Danh Mục Hồ Sơ Bàn Giao & Hướng Dẫn Vận Hành (Slide 40)

Toàn bộ hồ sơ kiểm thử, tài liệu báo cáo và mã nguồn đã được tổ chức hoàn chỉnh tại repository:

Hạng mục hồ sơ	Đường dẫn tệp / Thư mục	Nội dung kỹ thuật bàn giao
1. Mã nguồn	src/, package.json, tsconfig.json	Express 5, TypeScript ESM, Sequelize 6
2. Kiểm thử tự động	tests/unit/, tests/api/, jest.config.mjs	Jest, Supertest, ts-jest, tsconfig.spec.json
3. Kết quả test	docs/test-results.md	Ma trận đối chiếu chi tiết các kịch bản
4. Bảo mật API	docs/security-report.md + rate-limit	Helmet headers, loginLimiter HTTP 429
5. Tối ưu SQL	docs/sql-optimization.md	Số đo STATISTICS IO/TIME trước và sau
6. Docker & Compose	Dockerfile, .dockerignore, compose.yaml	Multi-stage build, healthcheck, named volume
7. Báo cáo triển khai	docs/deployment-report.md	Quy trình vận hành và kiểm chứng persistence
8. Postman Collection	docs/Chuong09-Test-Security-Docker.json	Toàn bộ request S01-S07, A01-A07, R01-R05

9. Báo cáo PDF & Word	docs/Bao_Cao_Thuc_Hanh_Chuong_09.pdf (.docx)	Báo cáo kỹ thuật chuẩn mực không tự chấm điểm

Bảng tổng hợp các lệnh kiểm thử và vận hành:

Mục đích kiểm thử	Lệnh thực thi dòng lệnh (Command)
Chạy toàn bộ Jest Test	pnpm test (hoặc node --experimental-vm-modules node_modules/jest/bin/jest.js)
Kiểm tra Test Coverage	pnpm test:coverage
Kiểm thử Live Security & Rate Limit	npx tsx src/scripts/run-chapter9-tests.ts
Đo lường SQL Server thực tế	npx tsx src/scripts/measure-sql.ts
Biên dịch toàn bộ mã nguồn	pnpm build && pnpm build:frontend

Báo cáo hoàn thành và được lưu trữ chính thức tại thư mục docs/.